Day 11 — Feature Engineering, Anomaly Detection, Explainability & Model Risk

Beginner-friendly summary

Today's core lesson is that **model quality starts before the model**.

Raw finance data such as:

```
transaction_date = 2026-05-14
amount           = 475000
vendor           = Vendor_1842
department       = Procurement
country          = IN
```

is rarely what a model should consume directly. We transform it into signals such as:

```
amount_vs_30d_avg
days_since_previous_payment
vendor_payment_count_before_today
amount_change_vs_previous_transaction
payment_hour
month_end_indicator
```

But there is a critical rule:

> **A feature for an event at time T may use only information that would have been available at time T.**

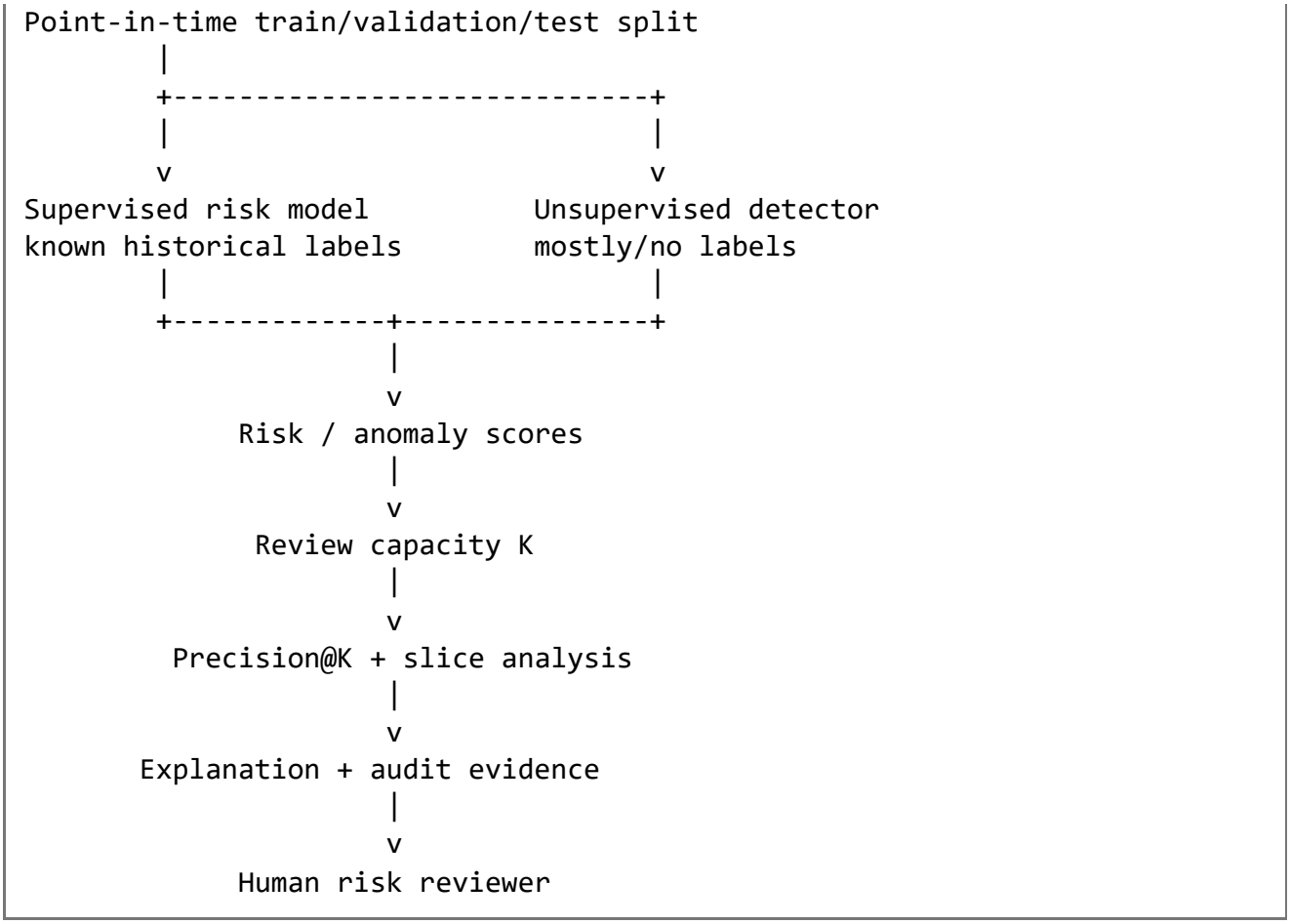
Otherwise we introduce leakage and create deceptively good offline results.

For risk systems, a second challenge appears: confirmed fraud/errors may be rare. A supervised model learns from known labels, while an anomaly detector asks, "Does this look unusual compared with normal behavior?" These systems solve related but different problems.

Finally, explanations such as SHAP, feature importance, and counterfactuals help us understand model behavior, but **an explanation is not proof of causality** and is not automatically sufficient for financial or regulatory decisions.

The senior-level mental model is:

```
Raw point-in-time data
        |
        v
Leakage-safe feature generation
        |
        +----> numerical / categorical / temporal
        |      aggregation / ratio / interaction
        |
        v
```



---

1. Finance feature engineering

Feature engineering converts raw observations into variables that expose useful patterns to a model.

A strong senior-level answer is not:

> “I create lots of features.”

It is:

> “I create point-in-time-correct features that represent economically meaningful behavior and remain stable enough to reproduce in production.”

Main feature families

Feature type	Finance example	Useful for
Numerical	amount, balance, credit limit	magnitude
Categorical	vendor, department, country	entity/context
Temporal	weekday, month-end, hour	recurring behavior
Aggregation	30-day average spend	behavioral history
Ratio	amount / historical average	relative deviation
Text-derived	invoice embedding/category	document information

Feature type	Finance example	Useful for
Interaction	amount × high-risk-country	combined effects

---

2. Numerical features

Examples:

```
transaction_amount
account_balance
invoice_amount
days_overdue
number_of_previous_payments
```

Raw values can sometimes be improved through transformations.

Log transformation

Financial amounts frequently have long-tailed distributions.

Instead of:

```
amount = 10
amount = 1_000
amount = 10_000_000
```

we might create:

```
amount_log = np.log1p(amount)
```

This is particularly useful for linear models because extreme values otherwise dominate relationships.

Trees generally tolerate skew much better.

---

3. Categorical features

Examples:

```
vendor
department
payment_method
currency
country
cost_center
```

The important issue is cardinality.

A column like:

```
payment_method
```

might contain five categories.

But:

```
vendor_id
```

might contain 700,000.

Those require different strategies.

---

4. Encoding categorical variables

One-hot encoding

For:

```
payment_method:

ACH
WIRE
CARD
```

we create:

```
payment_method_ACH
payment_method_WIRE
payment_method_CARD
```

Good when cardinality is relatively small.

Problem:

```
500,000 vendors
    ↓
potentially hundreds of thousands of columns
```

Ordinal encoding

Maps categories into integers:

```
ACH  -> 0
CARD -> 1
WIRE -> 2
```

The danger is that some models may interpret:

```
WIRE > CARD > ACH
```

even though there is no real ordering.

Tree models sometimes work acceptably with this representation, but arbitrary numerical ordering can still affect splits.

Frequency encoding

Replace the category with its historical frequency.

```
Vendor A appeared 1,200 times
Vendor B appeared 14 times
```

Features might become:

```
vendor_frequency = 1200
vendor_frequency = 14
```

This controls dimensionality.

But the frequency must be calculated using **historical information only**.

Target encoding

Suppose historical fraud rates are:

```
Vendor A -> 1.1%
Vendor B -> 8.7%
```

We could encode the vendor using those historical outcomes.

This can be powerful but is extremely leakage-prone.

Wrong:

```
fraud_rate = entire_dataset.groupby("vendor")["fraud"].mean()
```

The current row's own label contributes to its feature.

Correct approaches include:

```
out-of-fold encoding
historical encoding
regularized Bayesian encoding
```

For temporal finance problems, historical encoding is usually preferable.

---

5. Temporal features

A timestamp contains multiple potentially useful signals.

From:

```
2026-05-28 23:47:10
```

we might derive:

```
hour = 23
weekday = Thursday
month = May
quarter = Q2
is_weekend = False
is_month_end = True
```

Finance-specific examples are often stronger:

```
days_until_month_end
days_since_invoice
fiscal_quarter
payroll_week
days_since_previous_vendor_payment
days_since_account_creation
```

The business calendar can matter more than the normal calendar.

---

6. Aggregation features

Often some of the strongest finance features describe historical behavior.

Suppose an account normally spends:

```
₹20k
₹24k
₹19k
₹25k
```

and suddenly generates:

```
₹8,00,000
```

The raw amount is useful.

But this can be much stronger:

```
current_amount / historical_average
```

For example:

```
₹800,000 / ₹22,000 ≈ 36.4
```

That says:

> This transaction is approximately 36× this account's normal historical amount.

---

7. Lag-aware feature engineering

Consider transactions:

```
Time        Amount
-------------------
T1          100
T2          120
T3          90
T4          500
```

For the T4 prediction, this is valid:

```
mean(T1,T2,T3)
```

This is leakage:

```
mean(T1,T2,T3,T4)
```

because the aggregation includes the transaction currently being scored.

Even worse:

```
mean(T1...T10)
```

uses future transactions.

A common Pandas technique is:

```
df.groupby("account_id")["amount"].shift(1)
```

Then perform rolling calculations on the shifted series.

Conceptually:

```
shift first
   ↓
remove current observation
   ↓
rolling calculation
```

---

8. Ratios

Ratios convert absolute measurements into relative behavior.

Examples:

```
transaction_amount / historical_average_amount

outstanding_balance / credit_limit

actual_spend / approved_budget

invoice_amount / purchase_order_amount

vendor_payment / historical_vendor_payment
```

Ratios are especially useful when entities operate at very different scales.

A ₹100,000 transaction might be enormous for one small supplier and completely normal for another.

Important pitfalls

Protect against:

```
division by zero
tiny denominators
missing history
unstable ratios
```

Instead of blindly:

```
amount / historical_mean
```

use appropriate safeguards.

---

9. Interaction features

Sometimes two individually harmless variables become risky together.

For example:

```
large_amount = somewhat unusual
new_vendor   = somewhat unusual
```

Together:

```
large payment × new vendor
```

may be highly informative.

Explicit interaction:

```
large_new_vendor = large_payment * new_vendor
```

Linear models benefit strongly from explicitly constructed interactions.

Tree and boosted-tree models can learn many interactions automatically.

---

10. Text-derived features

Financial systems often contain:

```
invoice descriptions
expense descriptions
purchase-order text
audit notes
transaction narratives
```

Possible representations include:

```
TF-IDF
topic/category labels
sentiment-like indicators
keyword indicators
embeddings
LLM-extracted structured attributes
```

Example:

```
"urgent consulting payment offshore"
        ↓
embedding
        ↓
768-dimensional vector
```

For high-risk applications, I would generally prefer extracting stable, auditable structured signals when possible:

```
payment_purpose
contract_reference_present
invoice_number_present
vendor_name_match
```

rather than relying entirely on opaque text embeddings.

---

11. Scaling

Suppose features are:

```
amount             4,800,000
fraud_count        3
days_since_payment 12
```

Some algorithms depend heavily on feature magnitude.

Scaling matters for

```
Logistic regression
Linear regression with regularization
SVM
k-means
PCA
Neural networks
k-NN
```

Typical:

```
StandardScaler()
```

produces approximately:

```
mean = 0
std  = 1
```

Robust scaling based on medians and quantiles can be preferable when extreme outliers exist.

Why trees usually don't need scaling

A decision tree asks questions like:

```
amount < 150000?
```

Transforming the amount to:

```
amount_scaled < 0.72?
```

does not fundamentally change its ordering.

Therefore models such as:

```
DecisionTree
RandomForest
XGBoost
LightGBM
CatBoost
```

generally don't require StandardScaler-style scaling.

That can significantly simplify production feature pipelines.

---

12. PCA intuition

Suppose we have:

```
100 correlated numerical features
```

PCA rotates the feature space and creates combinations such as:

```
PC1
PC2
PC3
...
```

PC1 captures the largest direction of variation.

Conceptually:

```
x1 \
x2  \
x3   ---> PCA ---> PC1, PC2, PC3...
x4  /
x5 /
```

Instead of storing many correlated variables, perhaps:

```
100 features → 15 principal components
```

retain much of the variance.

When PCA helps

Potential situations include:

```
many correlated continuous measurements
very high-dimensional numerical data
distance-based algorithms
visualization
noise reduction
```

When PCA hurts

For financial risk systems, a reviewer understands:

```
payment_amount
vendor_age
budget_variance
```

but:

```
PC7 = 0.31x1 - 0.22x2 + 0.71x3 ...
```

is difficult to explain.

PCA can therefore trade:

```
dimensionality
      ↑
      |
explainability ↓
```

Also remember:

> PCA preserves variance, not predictive information.

A low-variance feature could still be highly predictive.

---

13. Clustering awareness

Clustering discovers structure without requiring outcome labels.

Typical use:

```
customer segmentation
vendor segmentation
spending behavior
branch segmentation
merchant behavior
```

K-means

Attempts to minimize within-cluster squared distance.

Conceptually:

```
Choose K centroids
      ↓
Assign each point to nearest centroid
      ↓
Recompute centroids
      ↓
Repeat
```

Strengths:

```
simple
fast
scalable
```

Weaknesses:

```
must select K
sensitive to scaling
sensitive to outliers
prefers roughly spherical clusters
```

Hierarchical clustering

Creates a tree-like hierarchy.

Useful when business users want different segmentation levels.

It becomes expensive for very large datasets.

DBSCAN

Groups dense regions.

Benefits:

```
doesn't require K
can find irregular cluster shapes
explicitly identifies noise points
```

Weaknesses:

```
epsilon difficult to tune
density varies across regions
performance deteriorates in high dimensions
```

Distance choice matters

K-means usually uses Euclidean distance.

But imagine:

```
amount       ₹1,000,000
risk_count   2
```

Without scaling, the amount dominates distance.

Other representations may require:

```
cosine distance → embeddings/text

Manhattan distance → some sparse/tabular situations

Gower-like distance → mixed categorical/numerical data
```

Distance is part of the model specification, not merely an implementation detail.

---

14. Unsupervised anomaly detection

An anomaly detector usually asks:

> How different is this observation from the distribution of normal historical behavior?

That is not the same as asking:

> Is this fraud?

An unusual legitimate transaction can be anomalous.

A common fraudulent pattern can eventually become statistically ordinary.

---

15. Isolation Forest

Isolation Forest has a particularly intuitive idea.

Normal observations live in crowded regions.

Strange observations can often be isolated with very few random splits.

```
Normal cluster:

 o o o o
o o o o o
 o o o o

                        X anomaly
```

`X` is relatively easy to isolate.

Advantages:

```
good general tabular baseline
scalable
```

```
doesn't require labeled anomalies
handles nonlinear structure
```

Limitations:

```
score not automatically a probability
contamination assumptions can mislead
unusual does not equal fraudulent
```

---

16. One-Class SVM

One-Class SVM tries to learn a boundary around normal observations.

Conceptually:

```
      normal region
    _______________
  /                 \
 |   x x x x x       |
 | x x x x x x       |
  \_________________/

               ? anomaly
```

Advantages:

```
flexible nonlinear boundaries
```

Disadvantages:

```
sensitive to scaling
sensitive to hyperparameters
can become expensive
less convenient on huge datasets
```

---

17. Robust statistics

Sometimes a sophisticated ML model isn't necessary.

Example:

```
median transaction = ₹20,000
MAD                = ₹4,000
transaction        = ₹200,000
```

Robust z-score-style detection can flag large deviations.

Why use robust statistics instead of mean/std?

Suppose:

```
10
11
12
12
13
9000
```

The mean and standard deviation are badly distorted.

Median and MAD remain much more stable.

For auditability, simple statistical detectors can be extremely valuable baselines.

---

18. Autoencoder awareness

An autoencoder learns:

```
Input
 ↓
Encoder
 ↓
compressed representation
 ↓
Decoder
 ↓
reconstructed input
```

Normal observations should reconstruct well.

Anomalies may have high:

```
reconstruction error
```

Useful for complex high-dimensional data.

But tabular finance systems should not automatically jump to neural autoencoders.

Costs include:

```
more training complexity
less explainability
greater tuning sensitivity
potential instability
harder model-risk review
```

Isolation Forest or robust statistical methods can often provide better operational trade-offs.

---

19. Evaluating anomaly detection with sparse labels

This is one of today's most important topics.

Suppose:

```
10,000,000 transactions/day
```

and operations can inspect:

```
500/day
```

Then ranking quality matters much more than generic classification accuracy.

Precision at review capacity

Suppose reviewers can inspect K=500 transactions.

Evaluate:

```
Precision@500
```

If:

```
80 confirmed relevant events
among 500 reviews
```

then:

```
Precision@500 = 80 / 500 = 16%
```

The specific number is only an example.

The key question is:

> Among cases operations actually have capacity to investigate, what fraction are useful?

Recall is still important

A detector might achieve:

```
high Precision@500
```

while missing entire fraud patterns.

Therefore monitor:

```
precision@K
recall@K where labels exist
PR-AUC
```

```
segment recall
false-negative investigation
```

---

20. Expert review

When labels are sparse, expert review becomes part of evaluation.

For example:

```
Top 200 anomalies
      ↓
risk analysts review
      ↓
confirmed issue
legitimate unusual activity
insufficient evidence
new pattern
```

Those results can later create higher-quality labels.

However, beware of the feedback loop:

```
model selects cases
      ↓
humans label only those cases
      ↓
training set represents model-selected population
```

You may never discover cases the model failed to surface.

Random sampling of some non-alerted transactions can reduce that blind spot.

---

21. Synthetic anomalies

Synthetic anomalies can test whether the pipeline detects known perturbations.

Examples:

```
100× normal transaction amount

sudden new-country payment

large amount + newly created vendor

10 payments within one minute
```

Useful for:

```
pipeline testing
sensitivity analysis
```

```
regression tests
```

But:

> Synthetic anomalies are not proof of real-world fraud performance.

They represent scenarios we imagined.

---

22. Backtesting

Finance models should generally be evaluated chronologically.

Example:

```
Train: Jan–Sep
Validate: Oct
Test: Nov

then

Train: Jan–Oct
Validate: Nov
Test: Dec
```

This tells us whether performance survives changing environments.

A random split might mix:

```
March behavior
December behavior
```

and hide temporal drift.

---

23. Imbalanced classification

Suppose:

```
fraud = 0.2%
normal = 99.8%
```

A model predicting:

```
everything = normal
```

achieves:

```
99.8% accuracy
```

and is useless.

---

24. Class weights

For logistic regression:

```
LogisticRegression(class_weight="balanced")
```

For tree models, equivalent weighted-loss mechanisms exist.

Conceptually:

```
mistake on rare positive
       ↓
larger penalty
```

Class weighting changes model training.

It does **not** magically create information about rare patterns.

---

25. Sampling

Undersampling

Reduce majority examples:

```
1,000,000 normal
2,000 fraud

        ↓

20,000 normal
2,000 fraud
```

Benefits:

```
faster training
better class balance
```

Cost:

```
discard information
```

Oversampling

Replicate or synthesize minority cases.

Methods include:

```
random oversampling
SMOTE-like approaches
```

For finance, synthetic sampling must be used carefully.

Creating mathematically plausible observations does not guarantee economically valid transactions.

Most importantly:

> Sampling occurs only inside the training partition.

Never modify validation/test distributions to make evaluation look balanced.

---

26. Focal loss concept

Normal cross entropy penalizes all misclassifications.

Focal loss reduces emphasis on easy examples and focuses learning on difficult ones.

Conceptually:

```
millions of easy normal cases
            ↓
     lower contribution

difficult rare cases
            ↓
     larger contribution
```

Common in deep learning.

For ordinary structured finance models, class weights and threshold optimization are often sufficient before introducing focal loss.

---

27. Threshold selection

A probability model might output:

```
0.12
0.41
0.63
0.91
```

The threshold doesn't have to be:

```
0.5
```

Suppose your risk team can review only:

```
300 cases/day
```

You can choose the threshold corresponding to approximately the top 300 scores.

That converts:

```
model scoring
```

into:

```
operational decision capacity
```

This distinction is important at senior level.

---

28. Feature selection

Why remove features?

Possible reasons:

```
noise
redundancy
latency
cost
instability
leakage risk
privacy
explainability
```

Methods include:

```
domain reasoning
univariate tests
regularization
permutation importance
tree importance
recursive elimination
stability analysis
```

The best feature is not merely predictive.

For production ML, a feature should ideally be:

```
predictive
available
point-in-time correct
stable
cheap enough
reproducible
governable
```

---

29. Collinearity

Suppose:

```
annual_salary
monthly_salary
weekly_salary
```

These contain nearly identical information.

For linear models, high collinearity can make coefficients unstable.

For example:

```
Run 1:
salary coefficient = +2.1
monthly_salary      = -0.7

Run 2:
salary coefficient = +0.4
monthly_salary      = +1.1
```

Predictions might remain stable while individual coefficients change dramatically.

That is dangerous if business users treat coefficients as explanations.

Tree models are less affected for predictive accuracy but importance can still be distributed unpredictably across correlated features.

---

30. Regularization

For linear models:

L1

Encourages coefficients to become exactly zero.

Useful for sparse feature selection.

L2

Shrinks coefficients toward zero.

Usually gives more stable models when correlated features exist.

Conceptually:

```
prediction loss
      +
complexity penalty
```

---

31. Feature drift

Suppose historical vendor payment amounts were:

```
₹10k–₹100k
```

but after a business acquisition:

```
₹100k–₹10M
```

Feature distributions have changed.

Monitor:

```
mean
median
quantiles
missingness
category frequency
unseen-category rate
PSI-like statistics
KS-like statistics
```

But:

> Feature drift does not automatically imply model failure.

It means investigation is warranted.

---

32. Feature stability

Ask:

```
Is the feature definition stable?

Does the upstream source change?

Does its meaning change?

Does availability change?

Does missingness change?

Does preprocessing remain identical?
```

A highly predictive unstable feature can be worse than a slightly weaker stable feature.

---

33. Explainability

There are two different questions.

Global

> What does the model generally depend upon?

Local

> Why did this particular transaction receive a high score?

Do not confuse them.

---

34. Permutation importance

Basic idea:

```
measure model performance
       ↓
shuffle feature X
       ↓
measure performance again
       ↓
large degradation means X was useful
```

Advantages:

```
model-agnostic
uses actual predictive effect
fairly intuitive
```

Problem with correlated features:

If:

```
x1 ≈ x2
```

shuffling `x1` might not hurt much because `x2` contains the same information.

Importance therefore appears artificially low.

---

35. SHAP intuition

SHAP asks approximately:

> Relative to a baseline prediction, how did each feature push this prediction higher or lower?

Example:

```
baseline risk                    0.03

large amount                    +0.08
new vendor                      +0.05
```

```
unusual payment hour             +0.02
long-standing customer           -0.03
                                ------
final model prediction            ...
```

The precise mathematics is based on Shapley values from cooperative game theory.

Local SHAP

For one transaction:

```
transaction #8172

amount_vs_history   +risk
vendor_age          +risk
known_department    -risk
```

Useful for case review.

Global SHAP

Aggregate SHAP magnitudes across many observations:

```
amount_vs_history
vendor_age
account_velocity
country_risk
...
```

This describes broad model behavior.

---

36. SHAP caveats

This is critical.

SHAP does **not** prove:

```
feature caused prediction outcome in real world
```

It explains:

```
feature contribution within this model
```

Problems include:

```
correlated features
unstable explanations
background dataset selection
```

```
feature interactions
model errors
proxy variables
```

A perfectly accurate explanation of a bad model is still a bad decision basis.

---

37. Counterfactual explanations

Suppose:

```
Current score = 0.78
```

A counterfactual asks:

> What small change would have lowered the score below the review threshold?

Example:

```
Current:

transaction amount = ₹900k
vendor age         = 1 day
payment hour       = 02:00

Counterfactual:

vendor age = 60 days
        ↓
risk score drops
```

But there is an important distinction.

Actionable vs non-actionable features

Potentially actionable:

```
requested payment amount
payment method
approval route
```

Usually non-actionable:

```
customer age
historical behavior
country of historical activity
past transactions
```

We should not tell someone:

> “Become older to reduce your model score.”

That is mathematically valid but operationally meaningless.

Counterfactual generation should constrain modifications to legitimate actionable variables.

---

38. Counterfactuals are not causal recommendations

Suppose:

```
Changing feature X would alter the model prediction.
```

That does not mean:

```
Changing X would alter actual fraud risk.
```

The model learned association, not necessarily causal structure.

This distinction matters especially when explanations are shown to decision-makers.

---

39. Slice-based error analysis

Overall performance may hide serious failures.

Suppose overall recall looks acceptable.

But examine:

```
country
department
vendor size
payment channel
transaction amount band
new vs established vendor
month
business unit
```

You might discover:

```
Established vendors → strong performance

New vendors → weak performance
```

The aggregate metric hid the dangerous slice.

---

40. Rare-event failures

Rare cases often matter most in finance.

For example:

```
international payments > ₹10M
```

could represent only 0.02% of transactions.

Overall metrics barely move if the model fails on every one.

Therefore examine:

```
high-value slices
rare categories
new categories
extreme amounts
new geographies
new products
```

Senior model evaluation is not merely:

> "PR-AUC increased."

It is:

> "Did the improvement hold on important operational and risk slices?"

Practical Task

We'll construct an **illustrative synthetic finance risk pipeline**.

The numerical outputs produced by the code are synthetic demonstration results, **not claimed production metrics**.

Our problem is:

```
Given a financial transaction,
rank transactions for risk review.
```

We compare:

Supervised

```
Logistic Regression
```

Why?

It gives us:

```
class weighting
probabilities/ranking
transparent coefficients
exact additive log-odds explanations
strong baseline
```

Unsupervised

```
Isolation Forest
```

Why?

It works without requiring fraud labels and is a strong general-purpose anomaly baseline.

Comparison and selection criteria

Method	Labels	Strength	Major weakness	Select when
Logistic regression	Required	Interpretable supervised baseline	Limited nonlinear interactions	Reliable labels exist
Random forest	Required	Nonlinear + interactions	Importance can mislead	Complex tabular relationships
Gradient boosting	Required	Usually strong tabular performance	More tuning	Predictive performance is primary
Isolation Forest	Not required	Practical anomaly baseline	Anomaly ≠ fraud	Labels are sparse
One-Class SVM	Mostly normal data	Flexible boundary	Scaling/tuning/scalability	Moderate-sized numerical data
Robust statistics	Not required	Transparent	Limited complexity	Auditable simple rules
Autoencoder	Not required	Complex representation	Explainability/operations	High-dimensional complex patterns

---

Design reasoning before implementation

Instead of optimizing immediately for maximum model complexity, I would establish these correctness conditions first.

1. Event time controls features

Every rolling statistic uses:

```
transactions strictly before the current transaction
```

2. Label time controls supervision

A transaction might occur today but fraud confirmation could arrive 45 days later.

The model cannot train on labels that were unavailable at the historical training cutoff.

Therefore distinguish:

```
event_time
```

from:

```
label_available_time
```

3. Preprocessors fit only on training data

This includes:

```
imputers
scalers
encoders
feature selectors
```

4. Validation/test preserve chronological ordering

No random train-test split.

5. Evaluation mirrors review capacity

If operations can inspect 100 cases:

```
Precision@100
```

is a critical metric.

6. Supervised and anomaly scores are not directly equivalent

Logistic score:

```
estimated supervised risk ranking
```

Isolation Forest score:

```
degree of statistical unusualness
```

Pseudocode

```
LOAD transactions
SORT chronologically

FOR each account:
    previous_amount = prior transaction
    rolling_mean = mean of PRIOR amounts only
    rolling_std = std of PRIOR amounts only
    transaction_velocity = PRIOR event count
    days_since_previous = current_time - previous_time

CREATE:
    amount_log
    amount_to_history_ratio
    temporal fields
```

```
    categorical fields

CREATE label_available_time

DEFINE temporal boundaries:
    training period
    validation period
    test period

TRAINING SUPERVISED DATA:
    event_time <= train_end
    AND label_available_time <= train_end

FIT preprocessing only on supervised training data
FIT weighted logistic regression

FIT anomaly preprocessor on training-period data
FIT Isolation Forest mostly on historical population

SCORE validation/test

FOR review capacity K:
    rank supervised scores
    calculate precision@K

    rank anomaly scores
    calculate precision@K

CALCULATE:
    PR-AUC where labels available
    slice metrics
    permutation importance

FOR individual transaction:
    calculate local feature contributions

DOCUMENT:
    training window
    label availability
    features
    exclusions
    thresholds
    limitations
    review process
```

Python implementation

```
import numpy as np
import pandas as pd

```

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler,
RobustScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import IsolationForest
from sklearn.metrics import average_precision_score, roc_auc_score
from sklearn.inspection import permutation_importance


# ------------------------------------------------------
# 1. Synthetic finance data
# ------------------------------------------------------

rng = np.random.default_rng(42)

n = 15_000

df = pd.DataFrame({
    "transaction_id": np.arange(n),
    "account_id": rng.integers(1, 500, n),
    "event_time": (
        pd.Timestamp("2025-01-01")
        + pd.to_timedelta(
            np.sort(rng.integers(0, 365 * 24 * 60, n)),
            unit="m"
        )
    ),
    "amount": rng.lognormal(mean=8.5, sigma=1.0, size=n),
    "vendor": rng.choice(
        [f"vendor_{i}" for i in range(150)],
        size=n
    ),
    "country": rng.choice(
        ["IN", "US", "GB", "SG", "DE"],
        size=n,
        p=[0.55, 0.15, 0.10, 0.10, 0.10]
    ),
    "payment_method": rng.choice(
        ["ACH", "WIRE", "CARD"],
        size=n,
        p=[0.55, 0.25, 0.20]
    )
})


# ------------------------------------------------------
# 2. Synthetic outcome
# ONLY for demonstrating the pipeline.
# These relationships are invented.
```

```
# ----------------------------------------------------

risk_signal = (
    (df["amount"] > df["amount"].quantile(0.97)).astype(int) * 1.5
    + (df["payment_method"] == "WIRE").astype(int) * 0.7
    + (df["country"] == "SG").astype(int) * 0.4
)

prob = 1 / (1 + np.exp(-(-4.5 + risk_signal)))

df["risk_label"] = rng.binomial(1, prob)


# Simulate delayed outcome confirmation
label_delay_days = rng.integers(5, 61, n)

df["label_available_time"] = (
    df["event_time"]
    + pd.to_timedelta(label_delay_days, unit="D")
)


# ----------------------------------------------------------
# 3. Leakage-safe historical features
# ----------------------------------------------------------

df = df.sort_values(
    ["account_id", "event_time", "transaction_id"]
).copy()


# Previous transaction
df["previous_amount"] = (
    df.groupby("account_id")["amount"]
      .shift(1)
)


# Previous timestamp
df["previous_event_time"] = (
    df.groupby("account_id")["event_time"]
      .shift(1)
)

df["days_since_previous"] = (
    (
        df["event_time"]
        - df["previous_event_time"]
    )
    .dt.total_seconds()
    .div(86400)
```

```
)


# Historical expanding mean:
# shift(1) ensures current transaction is NOT included.
df["historical_mean_amount"] = (
    df.groupby("account_id")["amount"]
      .transform(
          lambda s: s.shift(1).expanding().mean()
      )
)


df["historical_std_amount"] = (
    df.groupby("account_id")["amount"]
      .transform(
          lambda s: s.shift(1).expanding().std()
      )
)


# Ratio with denominator protection
safe_history = (
    df["historical_mean_amount"]
    .clip(lower=1.0)
)

df["amount_vs_history"] = (
    df["amount"] / safe_history
)


# Historical number of transactions
df["previous_transaction_count"] = (
    df.groupby("account_id")
      .cumcount()
)


# Temporal features
df["hour"] = df["event_time"].dt.hour
df["weekday"] = df["event_time"].dt.dayofweek
df["month"] = df["event_time"].dt.month
df["is_month_end"] = df["event_time"].dt.is_month_end.astype(int)

df["amount_log"] = np.log1p(df["amount"])


# Return to pure event-time order
df = df.sort_values("event_time").reset_index(drop=True)
```

```
# ---------------------------------------------------------
# 4. Chronological boundaries
# ---------------------------------------------------------

train_end = pd.Timestamp("2025-08-31 23:59:59")
valid_end = pd.Timestamp("2025-10-31 23:59:59")


# Supervised training requires BOTH:
# 1. event occurred before cutoff
# 2. label was available before cutoff

train_supervised = df[
    (df["event_time"] <= train_end)
    & (df["label_available_time"] <= train_end)
].copy()


valid = df[
    (df["event_time"] > train_end)
    & (df["event_time"] <= valid_end)
].copy()


test = df[
    df["event_time"] > valid_end
].copy()


# ---------------------------------------------------------
# 5. Feature definitions
# ---------------------------------------------------------

numeric_features = [
    "amount_log",
    "previous_amount",
    "historical_mean_amount",
    "historical_std_amount",
    "amount_vs_history",
    "previous_transaction_count",
    "days_since_previous",
    "hour",
    "weekday",
    "month",
    "is_month_end",
]

categorical_features = [
    "vendor",
    "country",
```

```
    "payment_method",
]


# ------------------------------------------------------
# 6. Supervised preprocessing
# ------------------------------------------------------

numeric_pipeline = Pipeline([
    (
        "imputer",
        SimpleImputer(strategy="median")
    ),
    (
        "scaler",
        StandardScaler()
    ),
])

categorical_pipeline = Pipeline([
    (
        "imputer",
        SimpleImputer(strategy="most_frequent")
    ),
    (
        "encoder",
        OneHotEncoder(
            handle_unknown="ignore"
        )
    ),
])

supervised_preprocessor = ColumnTransformer([
    (
        "num",
        numeric_pipeline,
        numeric_features
    ),
    (
        "cat",
        categorical_pipeline,
        categorical_features
    ),
])


supervised_model = Pipeline([
    (
        "preprocessor",
        supervised_preprocessor
    ),
```

```
    (
        "model",
        LogisticRegression(
            class_weight="balanced",
            max_iter=1000
        )
    ),
])


X_train = train_supervised[
    numeric_features + categorical_features
]

y_train = train_supervised["risk_label"]


supervised_model.fit(
    X_train,
    y_train
)


# ----------------------------------------------------------
# 7. Isolation Forest
# ----------------------------------------------------------

# Keep anomaly baseline transparent by using the
# numerical behavioral features.

anomaly_preprocessor = Pipeline([
    (
        "imputer",
        SimpleImputer(strategy="median")
    ),
    (
        "scaler",
        RobustScaler()
    ),
])


# Anomaly model can train on historical observations
# without needing confirmed positive labels.

anomaly_train = df[
    df["event_time"] <= train_end
]


X_anomaly_train = anomaly_preprocessor.fit_transform(
```

```
    anomaly_train[numeric_features]
)


isolation_forest = IsolationForest(
    n_estimators=300,
    contamination="auto",
    random_state=42,
)

isolation_forest.fit(X_anomaly_train)


# ------------------------------------------------------
# 8. Evaluation helpers
# ------------------------------------------------------

def precision_at_k(y_true, scores, k):
    k = min(k, len(y_true))

    ranked_indices = np.argsort(scores)[::-1][:k]

    return np.mean(
        np.asarray(y_true)[ranked_indices]
    )


def evaluate_partition(data, review_capacity=100):

    X = data[
        numeric_features + categorical_features
    ]

    y = data["risk_label"].to_numpy()

    supervised_scores = (
        supervised_model.predict_proba(X)[:, 1]
    )

    anomaly_X = anomaly_preprocessor.transform(
        data[numeric_features]
    )

    # score_samples:
    # higher = more normal
    #
    # negate so:
    # higher = more anomalous

    anomaly_scores = -isolation_forest.score_samples(
        anomaly_X
```

```
    )

    results = {
        "supervised_pr_auc":
            average_precision_score(
                y,
                supervised_scores
            ),

        "supervised_roc_auc":
            roc_auc_score(
                y,
                supervised_scores
            ),

        "supervised_precision_at_k":
            precision_at_k(
                y,
                supervised_scores,
                review_capacity
            ),

        "anomaly_precision_at_k":
            precision_at_k(
                y,
                anomaly_scores,
                review_capacity
            ),
    }

    return results


validation_results = evaluate_partition(
    valid,
    review_capacity=100
)

test_results = evaluate_partition(
    test,
    review_capacity=100
)


print("Illustrative synthetic validation:")
print(validation_results)

print("\nIllustrative synthetic test:")
print(test_results)
```

Do not interpret the printed numbers as expected production performance. They are simply the result of this invented synthetic experiment.

Why `shift(1)` is critical

This line deserves particular attention:

```
s.shift(1).expanding().mean()
```

Suppose:

```
amounts:

100
200
300
```

Without shift:

```
feature for transaction 3 =
mean(100,200,300)
```

The current transaction influences its own historical baseline.

With shift:

```
feature for transaction 3 =
mean(100,200)
```

which is point-in-time correct.

Another subtle correctness condition: delayed labels

This:

```
df["event_time"] <= train_end
```

is **not sufficient**.

Imagine:

```
transaction:
2025-08-20

fraud investigation completed:
2025-10-05
```

At an August 31 historical training cutoff, the August transaction existed.

Its fraud label did not.

Therefore:

```
df["label_available_time"] <= train_end
```

is also required.

This is an important form of label leakage that many pipelines miss.

Evaluating review capacity

Suppose operations can review:

```
100 transactions/day
```

Then both systems produce rankings:

```
Supervised:

transaction 82      0.97
transaction 912     0.94
transaction 184     0.92
...

Isolation Forest:

transaction 771     0.88 anomaly score
transaction 91      0.86
...
```

You inspect only:

```
top 100
```

and calculate:

```
precision_at_k(...)
```

This creates a much stronger connection between ML evaluation and business operations.

Global explanation using permutation importance

We can measure importance on an untouched validation set.

```
X_valid = valid[
    numeric_features + categorical_features
]
```

```
y_valid = valid["risk_label"]


importance = permutation_importance(
    supervised_model,
    X_valid,
    y_valid,
    scoring="average_precision",
    n_repeats=5,
    random_state=42,
)


importance_df = pd.DataFrame({
    "feature":
        numeric_features
        + categorical_features,

    "importance":
        importance.importances_mean,
})


importance_df = importance_df.sort_values(
    "importance",
    ascending=False
)

print(importance_df)
```

This answers approximately:

> Which original input fields matter most to validation ranking performance?

Not:

> Which variables causally create risk?

Local explanation for the logistic baseline

Logistic regression is particularly useful as an interpretable baseline because:

```
log odds
=
intercept
+
coefficient × transformed feature
+
...
```

We can therefore inspect contributions.

```
preprocessor = (
    supervised_model
    .named_steps["preprocessor"]
)

classifier = (
    supervised_model
    .named_steps["model"]
)


feature_names = (
    preprocessor
    .get_feature_names_out()
)


example = test.iloc[[0]][
    numeric_features + categorical_features
]


transformed = preprocessor.transform(example)


if hasattr(transformed, "toarray"):
    transformed = transformed.toarray()


values = transformed[0]

coefficients = classifier.coef_[0]

contributions = (
    values * coefficients
)


local_explanation = pd.DataFrame({
    "feature": feature_names,
    "value": values,
    "log_odds_contribution": contributions
})


local_explanation["absolute_contribution"] = (
    local_explanation[
        "log_odds_contribution"
    ].abs()
)
```

```
local_explanation = (
    local_explanation
    .sort_values(
        "absolute_contribution",
        ascending=False
    )
    .head(10)
)


print(local_explanation)
```

This generates an actual model explanation rather than inventing one.

Using SHAP in a stronger tree model

If later we replace the supervised baseline with something like:

```
XGBoost
LightGBM
CatBoost
Random Forest
```

SHAP becomes particularly useful.

Conceptually:

```
import shap

explainer = shap.Explainer(model, background_data)

shap_values = explainer(rows_to_explain)

shap.plots.waterfall(
    shap_values[0]
)
```

I would treat SHAP as part of the explanation layer, not as validation that the underlying model is correct.

Example counterfactual workflow

Suppose a case has:

```
risk_score = 0.81
review_threshold = 0.70
```

Generate plausible changes only for permitted variables.

Conceptual pseudocode:

```
original transaction

FOR each actionable feature:
    generate valid candidate values

    KEEP:
        all immutable features unchanged

    SCORE candidate

    IF score < threshold:
        calculate distance from original

RETURN smallest valid modification
```

Example result might say:

```
The model score would fall below the review threshold
if the requested payment were <= ₹X,
holding all other modeled variables constant.
```

But the reviewer documentation should explicitly state:

> This is a model counterfactual, not evidence that reducing the payment would causally reduce underlying risk.

Slice-based evaluation

Add something like:

```
def slice_report(
    data,
    slice_column,
    minimum_rows=50,
    review_capacity=50
):

    reports = []

    for slice_value, group in data.groupby(slice_column):

        if len(group) < minimum_rows:
            continue

        y = group["risk_label"].to_numpy()

        X = group[
            numeric_features + categorical_features
```

```
        ]

        scores = (
            supervised_model
            .predict_proba(X)[:, 1]
        )

        reports.append({
            "slice": slice_value,
            "rows": len(group),
            "positive_rate": y.mean(),
            "precision_at_k":
                precision_at_k(
                    y,
                    scores,
                    review_capacity
                )
        })

    return pd.DataFrame(reports)


print(
    slice_report(
        test,
        "country"
    )
)
```

Other slices worth testing:

```
new vs existing accounts
high vs low value
wire vs card
region
business unit
vendor tenure
month
known vs unseen categories
```

What could invalidate this practical experiment?

Several things.

1. Synthetic data is unrealistic

Our generated outcome intentionally follows a simple invented relationship.

Real financial risk contains:

```
adversarial behavior
complex interactions
changing processes
label noise
investigation bias
```

Therefore this experiment proves pipeline mechanics, not real-world effectiveness.

---

2. Investigation labels may be biased

Fraud labels often exist disproportionately for:

```
previously alerted cases
high-value cases
specific regions
known fraud patterns
```

The supervised dataset can therefore represent:

```
what we historically investigated
```

rather than:

```
all fraud in the population
```

---

3. Historical aggregates may differ online

Offline:

```
Pandas dataframe
```

Online:

```
feature store / database / stream
```

If definitions differ, training-serving skew appears.

Point-in-time correctness must be reproduced in serving.

---

4. Fraud strategies evolve

A feature effective six months ago can become useless once behavior changes.

Backtesting and temporal monitoring therefore matter.

---

5. Precision can hide coverage failure

High:

```
Precision@100
```

could occur while missing an entire emerging fraud class.

Review:

```
rare-event slices
false negatives
random non-alert sample
```

as well.

Production design decisions

For a production implementation, I would separate features into three groups.

Online-safe

Available immediately:

```
transaction amount
timestamp
payment method
vendor identifier
account identifier
```

Historical online features

Require low-latency state:

```
30-day transaction count
previous amount
vendor historical frequency
account historical average
```

These may come from:

```
feature store
stream state
low-latency database
```

Offline-only features

Examples:

```
investigation result
future settlement status
chargeback received later
auditor disposition
```

These must never enter real-time scoring features.

They may be labels instead.

Model-risk note for a finance reviewer

Here is how I would document the illustrative system.

Purpose

The system ranks financial transactions for human risk review. It does not autonomously determine fraud, approve/reject transactions, or replace investigator judgment.

Models

The supervised component uses historical confirmed outcomes to estimate relative transaction risk. An Isolation Forest independently identifies statistically unusual transactions without relying on confirmed outcome labels.

Feature controls

All behavioral aggregates are generated using transactions available strictly before the scored event. Current and future transactions are excluded from historical aggregates. Supervised training additionally excludes labels that were unavailable at the historical training cutoff.

Evaluation

The primary operational evaluation includes precision among the highest-ranked transactions up to available reviewer capacity. PR-AUC and temporal backtesting provide additional model-quality evidence. Performance should also be evaluated across material business slices and rare high-impact scenarios.

Interpretation

Feature importance and local explanations describe model behavior rather than causal relationships. An anomaly score indicates unusualness and should not be interpreted as a probability of fraud.

Human oversight

Alerts require human review. Reviewers should have access to underlying transaction evidence rather than relying solely on model explanations.

Key limitations

Important risks include delayed or biased labels, previously unseen fraud patterns, population drift, new vendors/categories, upstream feature changes, correlated features, and investigation-selection bias.

Monitoring

Production monitoring should include score distributions, feature drift, missingness, unseen categories, reviewer yield, Precision@K when labels mature, slice performance, alert

volumes, latency, and training-serving feature consistency.

Senior-level distinctions to remember

These distinctions are especially important:

Common confusion	Correct distinction
Anomaly = fraud	Anomaly means unusual
SHAP = causality	SHAP explains model contribution
High overall metric = safe model	Important slices can still fail
Random split = adequate validation	Temporal finance data usually requires chronological validation
Event exists = label exists	Labels can arrive much later
Historical average = safe	Only if future/current rows are excluded
0.5 = correct threshold	Threshold should reflect business cost/capacity
PCA = better features	PCA preserves variance, not necessarily predictive value
More features = better	Stability and point-in-time availability also matter
Accuracy = good imbalance metric	Rare-event systems require PR/ranking analysis

---

Day 11 core mental model

The best way to connect today's topics is:

```
Feature engineering
      ↓
"What did we know at prediction time?"

Modeling
      ↓
"What pattern can we reliably learn?"

Anomaly detection
      ↓
"What unusual cases might labels miss?"

Evaluation
      ↓
"What can reviewers actually investigate?"

Explainability
      ↓
"What drove the MODEL'S prediction?"

Counterfactual
```

```
      ↓
"What modeled change would alter the score?"

Model risk
      ↓
"Where can this entire reasoning process fail?"
```

The senior-level principle is:

> **A finance ML system is not reliable merely because its model scores well. It is reliable when features are point-in-time correct, labels are historically available, validation reflects future deployment, alert volumes fit operational capacity, explanations are presented with proper limitations, important slices are tested, and the entire decision path can be reconstructed for review.**

Day 11 DSA — BFS

1. Beginner-friendly summary

Breadth-First Search (BFS) explores nodes in increasing distance from a starting point.

Its defining data structure is a **FIFO queue**:

```
Start
  |
  v
Level 0:        A
               / \
Level 1:      B   C
             / \   \
Level 2:    D  E   F
```

BFS processes:

```
A
B, C
D, E, F
```

This makes BFS especially useful for:

- tree level-order traversal,
- graph traversal,
- shortest paths in **unweighted** graphs,
- minimum moves/steps/hops,
- grids where every move has equal cost.

The key rule:

> When all edges have the same cost, the first time BFS reaches a node is through a shortest path from the source.

---

2. Recognition signals

Think **BFS** when the problem contains phrases like:

```
minimum number of steps
shortest path
fewest moves
nearest
minimum transformations
level by level
all nodes at distance K
spread one step per minute
```

Typical problems include:

```
Tree level-order traversal
Shortest path in an unweighted graph
Shortest path in a grid
Word Ladder
Rotting Oranges
Minimum knight moves
Nearest exit
Minimum genetic mutations
```

A strong recognition pattern is:

```
State
 +
Transitions between states
 +
Each transition has equal cost
 +
Need minimum number of transitions
        ↓
       BFS
```

---

3. BFS versus DFS

Suppose:

DFS might explore:

```
A → B → D → F
```

before looking at `C`.

BFS explores:

```
A
B C
D E
F
```

Therefore if `E` is the target:

```
A → C → E
```

BFS discovers that short path before exploring very deep alternatives.

Quick comparison

Requirement	BFS	DFS
Traverse graph	Yes	Yes
Tree level order	**Best fit**	Awkward
Unweighted shortest path	**Best fit**	No guarantee
Explore deeply	No	**Yes**
Queue	**Yes**	No
Stack/recursion	No	**Yes**

---

4. The BFS queue invariant

This is the most important conceptual point.

At any moment, BFS's queue contains the **frontier** of discovered but not-yet-processed nodes.

Suppose:

```
A → B → D
 \
  → C → E
```

Starting with `A`:

```
Queue: [A]
```

Process `A`:

```
Queue: [B, C]
```

Both are distance 1.

Process `B`:

```
Queue: [C, D]
```

Distances are:

```
C = 1
D = 2
```

Process `C`:

```
Queue: [D, E]
```

Both `D` and `E` are distance 2.

The queue therefore maintains:

> Nodes are processed in non-decreasing shortest-path distance from the source.

That's why BFS gives shortest paths for equal-cost edges.

---

5. Critical invariant: mark visited when enqueuing

Consider:

```
   A
  / \
 B   C
  \ /
   D
```

When processing `B`, we discover `D`.

Correct:

```
visited.add(D)
queue.append(D)
```

If you wait until `D` is removed from the queue before marking it visited, `C` could also enqueue `D`.

You might get:

```
Queue:
```

```
D
D
```

In larger graphs this can explode.

Therefore:

> **Discover → mark visited → enqueue**

not:

```
discover
enqueue
...
eventually mark visited
```

---

6. Basic graph BFS

Given:

```
graph = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F"],
    "D": [],
    "E": [],
    "F": [],
}
```

BFS:

```
from collections import deque


def bfs(graph, start):
    queue = deque([start])
    visited = {start}

    while queue:
        node = queue.popleft()

        print(node)

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
```

Produces:

```
A
B
C
D
E
F
```

---

7. Why `deque` instead of a Python list?

Avoid:

```
queue = []
queue.pop(0)
```

Removing the first element of a Python list costs:

```
O(n)
```

because remaining elements shift.

Use:

```
from collections import deque

queue.popleft()
```

which is:

```
O(1)
```

So the standard BFS queue is:

```
queue = deque()
```

---

8. Tree level-order BFS

Suppose:

```
        3
       / \
      9   20
         /  \
        15   7
```

Expected:

```
[
    [3],
    [9, 20],
    [15, 7]
]
```

The important technique is:

```
level_size = len(queue)
```

At the beginning of each iteration, everything already in the queue belongs to the current level.

```
from collections import deque


def level_order(root):
    if root is None:
        return []

    queue = deque([root])
    result = []

    while queue:
        level_size = len(queue)
        level = []

        for _ in range(level_size):
            node = queue.popleft()
            level.append(node.val)

            if node.left:
                queue.append(node.left)

            if node.right:
                queue.append(node.right)

        result.append(level)

    return result
```

Why snapshot `len(queue)`?

Because while processing the current level, its children are added to the queue.

If you dynamically processed everything until the queue became empty, you'd mix multiple levels together.

---

9. Shortest path in an unweighted graph

Consider:

```
A ---- B ---- D
|      |
|      |
C ---- E ---- F
```

Every edge has cost `1`.

We want the shortest distance from:

```
A → F
```

BFS can store:

```
(node, distance)
```

```
from collections import deque


def shortest_distance(graph, start, target):
    queue = deque([(start, 0)])
    visited = {start}

    while queue:
        node, distance = queue.popleft()

        if node == target:
            return distance

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, distance + 1))

    return -1
```

The moment `target` is removed from the BFS queue, we know its distance is minimum.

---

10. When BFS does **not** give the correct shortest path

Suppose edges have different costs:

```
A --100--> B

A --1--> C --1--> B
```

BFS thinks in:

```
number of edges
```

rather than:

```
total edge weight
```

So normal BFS is appropriate for:

```
unweighted graph

or

every edge has the same cost
```

For weighted non-negative edges, think:

```
Dijkstra
```

For negative weights, other algorithms such as Bellman-Ford may be needed.

Medium Problem

Shortest Path in Binary Matrix

Given an `n × n` binary matrix:

```
0 = open cell
1 = blocked cell
```

Start at:

```
(0, 0)
```

and reach:

```
(n-1, n-1)
```

You may move in eight directions:

```
↖ ↑ ↗
← X →
↙ ↓ ↘
```

Return the length of the shortest clear path.

If no path exists, return:

```
-1
```

Example:

```
grid:

0 1 0
0 0 0
1 0 0
```

One shortest path is:

```
(0,0)
   ↓ diagonal
(1,1)
   ↓ diagonal
(2,2)
```

Path length:

```
3
```

---

11. Recognition signals

Before coding, recognize:

State

A cell:

```
(row, col)
```

Transition

Move to any valid neighboring cell.

Cost

Every move costs exactly:

```
1
```

Objective

Find:

```
shortest path
```

Therefore:

```
Grid
+
equal-cost moves
+
shortest path
        ↓
       BFS
```

---

12. Brute-force reasoning

A naive approach would generate **every possible path** from `(0,0)` to `(n-1,n-1)`.

For every cell, potentially try up to eight choices:

```
                  start
          / / / / | \ \ \ \
         ...
       / / / /
      ...
```

Some paths:

- revisit cells,
- form cycles,
- reach dead ends,
- overlap heavily.

The number of possible paths can grow exponentially.

Rough conceptual behavior:

```
O(8^k)
```

for path length `k`, ignoring boundaries.

This is unacceptable.

---

13. Why BFS is optimized

Instead of storing complete alternative paths, BFS explores:

```
distance 1
distance 2
distance 3
...
```

Each reachable cell needs to be processed at most once.

For an `n × n` matrix:

```
number of cells = n²
```

Each cell checks at most:

```
8 neighbors
```

Therefore:

```
O(8 × n²)
=
O(n²)
```

---

14. Optimized reasoning

Initialize:

```
queue = [(0, 0, 1)]
```

where:

```
row = 0
col = 0
distance = 1
```

Then repeatedly:

```
remove nearest cell

if target:
    return distance

for 8 neighbors:
    if inside matrix
       and open
       and not visited:

        mark visited
        enqueue with distance + 1
```

Because BFS processes nodes by increasing distance:

> The first time we reach the destination gives a shortest path.

---

15. Edge cases before coding

Empty grid

Potential defensive case:

```
[]
```

Return:

```
-1
```

Starting cell blocked

```
1 ?
? ?
```

Impossible.

Return:

```
-1
```

Destination blocked

```
? ?
? 1
```

Impossible.

Single cell

```
0
```

Start is already destination.

Answer:

```
1
```

No path

```
0 1 1
1 1 1
1 1 0
```

Return:

```
-1
```

Diagonal path

Remember there are eight directions, not four.

---

16. Complexity

There are:

```
n² cells
```

Each is processed at most once.

Each processing examines eight neighbors.

Time

```
O(n²)
```

Space

The queue plus visited information may contain:

```
O(n²)
```

cells.

So:

```
Space = O(n²)
```

---

17. Pseudocode

```
IF grid is empty:
    RETURN -1

IF start blocked OR destination blocked:
    RETURN -1

directions = 8 possible moves

queue = [(0, 0, 1)]

mark start visited

WHILE queue not empty:

    row, col, distance = dequeue

    IF row,col is destination:
        RETURN distance

    FOR each direction:

        new_row = row + dr
```

```
        new_col = col + dc

        IF:
            new position inside grid
            AND cell is open
            AND cell not visited

            mark visited

            enqueue(
                new_row,
                new_col,
                distance + 1
            )

RETURN -1
```

---

18. Python solution

```
from collections import deque
from typing import List


def shortest_path_binary_matrix(grid: List[List[int]]) -> int:
    if not grid or not grid[0]:
        return -1

    n = len(grid)

    if grid[0][0] != 0 or grid[n - 1][n - 1] != 0:
        return -1

    directions = [
        (-1, -1),
        (-1,  0),
        (-1,  1),
        ( 0, -1),
        ( 0,  1),
        ( 1, -1),
        ( 1,  0),
        ( 1,  1),
    ]

    queue = deque([
        (0, 0, 1)
    ])

    visited = {(0, 0)}

    while queue:
```

```
        row, col, distance = queue.popleft()

        if row == n - 1 and col == n - 1:
            return distance

        for dr, dc in directions:
            new_row = row + dr
            new_col = col + dc

            inside_grid = (
                0 <= new_row < n
                and 0 <= new_col < n
            )

            if not inside_grid:
                continue

            if grid[new_row][new_col] != 0:
                continue

            if (new_row, new_col) in visited:
                continue

            visited.add(
                (new_row, new_col)
            )

            queue.append(
                (
                    new_row,
                    new_col,
                    distance + 1,
                )
            )

    return -1
```

---

19. Walkthrough

Input:

```
0 1 0
0 0 0
1 0 0
```

Initialize:

```
queue:
```

```
[(0,0,1)]
```

Visited:

```
(0,0)
```

Process:

```
(0,0,1)
```

Reachable neighbors include:

```
(1,0)
(1,1)
```

Queue:

```
[(1,0,2), (1,1,2)]
```

Process `(1,0)`.

Then process:

```
(1,1,2)
```

It can reach:

```
(2,2,3)
```

Eventually:

```
queue → (2,2,3)
```

Destination found.

Return:

```
3
```

---

20. A useful optimization: reuse the grid as visited state

Instead of:

```
visited = {(0, 0)}
```

we can modify the input:

```
grid[0][0] = 1
```

Then whenever we discover an open cell:

```
grid[new_row][new_col] = 1
```

This converts:

```
0 = available
1 = unavailable / visited / blocked
```

and avoids a separate set.

```
from collections import deque
from typing import List


def shortest_path_binary_matrix(grid: List[List[int]]) -> int:
    if not grid or not grid[0]:
        return -1

    n = len(grid)

    if grid[0][0] != 0 or grid[n - 1][n - 1] != 0:
        return -1

    directions = [
        (-1, -1), (-1, 0), (-1, 1),
        (0, -1),           (0, 1),
        (1, -1),  (1, 0),   (1, 1),
    ]

    queue = deque([(0, 0, 1)])

    grid[0][0] = 1

    while queue:
        row, col, distance = queue.popleft()

        if row == n - 1 and col == n - 1:
            return distance

        for dr, dc in directions:
            nr = row + dr
            nc = col + dc

            if (
                0 <= nr < n
                and 0 <= nc < n
                and grid[nr][nc] == 0
```

```
            ):
                grid[nr][nc] = 1
                queue.append(
                    (nr, nc, distance + 1)
                )

    return -1
```

Trade-off

This saves the explicit visited structure but modifies the caller's input.

That can be undesirable when:

```
grid must be reused
caller expects immutability
debugging requires original input
```

So this is a design choice, not universally an improvement.

---

21. Another BFS pattern: process by levels

Instead of storing:

```
(row, col, distance)
```

with every queue element, you can let queue boundaries represent distance.

```
queue = deque([(0, 0)])
distance = 1

while queue:
    level_size = len(queue)

    for _ in range(level_size):
        row, col = queue.popleft()

        ...

    distance += 1
```

This pattern is especially useful for questions phrased as:

```
minimum minutes
minimum transformations
number of levels
minimum moves
```

---

22. Parent pointers when you need the actual path

Sometimes returning:

```
3
```

isn't enough.

You may need:

```
[(0,0), (1,1), (2,2)]
```

Instead of copying the entire path into every queue entry, store:

```
parent[child] = current
```

Example:

```
parent[(1,1)] = (0,0)
parent[(2,2)] = (1,1)
```

When destination is reached:

```
destination
     ↓
parent
     ↓
parent
     ↓
source
```

Then reverse the reconstructed path.

This is usually more memory-efficient than putting entire paths into the queue.

---

23. Common BFS mistakes

Mistake 1 — Using DFS for shortest unweighted path

```
dfs(...)
```

can find **a** path, but not necessarily the shortest path.

Mistake 2 — Marking visited too late

Bad:

```
node = queue.popleft()
visited.add(node)
```

Multiple parents may already have queued that node.

Better:

```
visited.add(neighbor)
queue.append(neighbor)
```

Mistake 3 — Forgetting cycles

A graph might contain:

```
A → B → C
↑       |
└_______┘
```

Without visited tracking:

```
A → B → C → A → B → C ...
```

Mistake 4 — Using `pop(0)`

Avoid:

```
queue.pop(0)
```

Use:

```
deque.popleft()
```

Mistake 5 — Assuming BFS handles arbitrary weighted edges

BFS optimizes:

```
number of edges
```

not arbitrary total cost.

Mistake 6 — Incorrect level handling

Don't do:

```
for _ in range(len(queue)):
```

if the relevant queue length could conceptually be reevaluated while children are inserted.

Instead snapshot it clearly:

```
level_size = len(queue)

for _ in range(level_size):
```

---

24. BFS complexity template

For a normal graph:

```
V = vertices
E = edges
```

Each vertex is visited once and each adjacency is inspected.

Therefore:

```
Time  = O(V + E)
Space = O(V)
```

For a grid:

```
rows = R
cols = C
```

There are:

```
R × C
```

possible states.

Therefore typically:

```
Time  = O(R × C)
Space = O(R × C)
```

assuming a constant number of possible moves from each cell.

---

25. BFS pattern template to remember

```
from collections import deque


def bfs(start):
    queue = deque([start])
    visited = {start}

    while queue:
        current = queue.popleft()
```

```
        if is_target(current):
            return current

        for neighbor in get_neighbors(current):
            if neighbor in visited:
                continue

            visited.add(neighbor)
            queue.append(neighbor)
```

For shortest distance:

```
queue = deque([
    (start, 0)
])
```

For tree levels:

```
level_size = len(queue)
```

For recovering a path:

```
parent[neighbor] = current
```

These are four variations of the same underlying BFS idea.

---

26. Final recognition framework

When you see a new problem, mentally run:

```
1. What is a state?
        ↓
   node / cell / word / configuration

2. What are the neighbors?
        ↓
   edges / moves / transformations

3. Are transitions equally expensive?
        ↓
      YES

4. Do I need minimum moves/distance
   or level-order exploration?
        ↓
      YES

5. Can states repeat?
        ↓
```

```
   maintain visited

6. Need actual route?
        ↓
   maintain parent pointers

              ↓

             BFS
```

For this medium problem:

```
State       = matrix cell
Neighbor    = eight adjacent cells
Edge cost   = 1
Goal        = minimum path length
Cycle risk  = yes
Algorithm   = BFS
Time        = O(n²)
Space       = O(n²)
```

The single most important BFS invariant to retain is:

> **Once an unvisited node is discovered, mark it visited immediately and enqueue it. Because the FIFO queue processes states in non-decreasing distance order, the first discovery of a node represents a shortest path in an unweighted graph.**